

1. What is a View?

A **View** is a virtual table based on the result-set of a SQL statement.

- It contains rows and columns just like a real table.
- It does **not** store data physically on the disk (with some exceptions in other DBs, but in MySQL, it's a virtual structure).
- It fetches data dynamically from the underlying source tables (called **base tables**) every time you query it.

Why Use Views?

- **Simplicity:** Simplifies complex queries (like multi-table joins) into a single `SELECT * FROM view_name` statement.
 - **Security:** Restricts user access to specific rows or columns without exposing the entire base table.
 - **Consistency:** Provides a consistent, unchanging legacy structure even if the underlying table layouts change.
-

2. Setup (Sample Tables)

To practice the examples below, assume we have the following two tables: employees and departments.

employees Table:

emp_id	name	role	salary	dept_id
1	Alice	Developer	85000	101
2	Bob	Designer	70000	102
3	Charlie	Manager	95000	101

departments Table:

dept_id	dept_name
---------	-----------

dept_id	dept_name
101	IT
102	Creative

3. Creating and Querying a View

Syntax

SQL

```
CREATE VIEW view_name AS
```

```
SELECT column1, column2, ...
```

```
FROM table_name
```

```
WHERE condition;
```

Example 1: Simple View (Data Masking/Security)

Let's create a view that shows employee names and roles but hides their sensitive salary information.

SQL

```
CREATE VIEW employee_public_info AS
```

```
SELECT emp_id, name, role
```

```
FROM employees;
```

Querying the view:

You query a view exactly like a normal table.

SQL

```
SELECT * FROM employee_public_info;
```

Output:

emp_id	name	role
--------	------	------

emp_id	name	role
1	Alice	Developer
2	Bob	Designer
3	Charlie	Manager

Example 2: Complex View (Simplifying Joins)

Let's create a view that combines employee details with their respective department names.

SQL

```
CREATE VIEW employee_dept_details AS
SELECT e.emp_id, e.name, e.role, d.dept_name
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id;
```

Querying the view:

SQL

```
SELECT * FROM employee_dept_details WHERE dept_name = 'IT';
```

4. Modifying and Dropping Views

Replacing or Altering a View

If you want to change the query inside an existing view, you can use CREATE OR REPLACE VIEW or ALTER VIEW.

SQL

```
CREATE OR REPLACE VIEW employee_public_info AS
SELECT emp_id, name, role, dept_id
FROM employees;
```

Dropping a View

To remove a view permanently from the database (this does **not** delete data from the base tables):

SQL

```
DROP VIEW IF EXISTS employee_public_info;
```

5. Updatable Views (INSERT, UPDATE, DELETE)

In MySQL, some views are **updatable**. This means you can run UPDATE, INSERT, or DELETE statements on the view, and the changes will be applied directly to the underlying base table.

Requirements for an Updatable View

For a view to be updatable, there must be a **1-to-1 relationship** between the rows in the view and the rows in the base table. The view query **cannot** contain any of the following:

- Aggregate functions (SUM(), AVG(), COUNT(), etc.)
- DISTINCT
- GROUP BY or HAVING
- UNION or UNION ALL
- Joins (MySQL allows updates on some joined views, but inserts are generally restricted)

Example: Updating through a View

SQL

```
-- Updating Bob's role using the view
```

```
UPDATE employee_public_info
```

```
SET role = 'Senior Designer'
```

```
WHERE name = 'Bob';
```

Note: If you check the underlying employees table, Bob's role will now be updated to "Senior Designer".

6. The WITH CHECK OPTION Clause

When creating an updatable view, you can add WITH CHECK OPTION at the end. This prevents you from inserting or updating rows through the view that do not satisfy the view's WHERE clause conditions.

Example:

SQL

```
CREATE VIEW high_earners AS
```

```
SELECT emp_id, name, salary
```

```
FROM employees
```

```
WHERE salary >= 80000
```

```
WITH CHECK OPTION;
```

- **Allowed:** UPDATE high_earners SET salary = 85000 WHERE emp_id = 1; (Salary stays above 80k).
- **Blocked with an Error:** UPDATE high_earners SET salary = 50000 WHERE emp_id = 1; (This would make the row disappear from the view, so MySQL blocks it).

7. Quick Summary Cheat Sheet

Feature	Description
Storage	Views do not store data; they store the underlying SELECT query definition.
Performance	Because they run the query every time, poorly written views over huge datasets can slow things down.
Security benefit	Hides specific rows/columns from certain users.
Dropping	DROP VIEW view_name; does not touch actual data.